

Zomato ETL Implementation on Databricks

Using Unity Catalog, Delta Lake & Structured Notebook Orchestration

1. Introduction

1.1 Project Overview

The **Zomato ETL Implementation Project** focuses on designing and implementing a **production-grade ETL (Extract, Transform, Load) pipeline** on **Databricks Free Edition**, adhering to modern **data engineering best practices**.

The pipeline ingests **semi-structured JSON data** from Zomato restaurant metadata, transforms it into structured formats, and loads it into **Delta Lake tables managed by Unity Catalog**. The final output enables **analytics-ready, partitioned summary tables** suitable for reporting and downstream consumption.

This project emphasizes not just data movement, but also:

- Data quality enforcement
 - Schema governance
 - Audit logging
 - Incremental & historical loading
 - Operational safety via orchestration logic
-

1.2 Objectives of the Project

The primary objectives of this project are:

- Implement a **multi-stage ETL pipeline** completely within Databricks
 - Use **Unity Catalog** for centralized governance and table management
 - Apply **Delta Lake** for ACID compliance and scalable storage
 - Follow the **Medallion Architecture** (Bronze → Silver → Gold)
 - Support **manual and historical loading strategies**
 - Ensure **traceability, logging, and auditability**
 - Design for **daily scheduled execution**
-

1.3 Key Technologies Used

Technology	Purpose
Databricks Free Edition	ETL execution platform
Unity Catalog	Centralized data governance
Delta Lake	ACID tables & scalable storage
Databricks Volumes	Managed file storage
PySpark & Spark SQL	Data transformations
Structured Notebooks	Orchestration and modular execution

1.4 Medallion Architecture Adoption

The solution strictly follows the **Medallion Architecture**, ensuring separation of concerns and data quality at each stage.

Layer	Description	Implementation
Bronze	Raw structured ingestion	zomato_raw
Silver	Cleaned & enriched data	Summary staging
Gold	Analytics-ready aggregates	zomato_summary

This layered approach enables:

- Reprocessing flexibility
 - Clear transformation lineage
 - Easier debugging and auditing
-

1.5 High-Level ETL Flow

1. **Ingestion**
 - Raw JSON files arrive in Databricks Volumes
2. **Processing**
 - JSON → CSV conversion

- Raw Delta loading
- Dimension enrichment

3. Transformation

- Filtering
- Deduplication
- Standardization
- Business metrics derivation

4. Serving

- Partitioned summary tables
- Query-optimized Delta Lake format

1.6 Architecture Overview

The ETL workflow is implemented as **three core processing modules**, driven by a central **Driver Notebook**.

Modules

1. Module 1 – JSON to CSV Conversion

- Semi-structured → structured format

2. Module 2 – Raw & Dimension Table Creation

- Delta ingestion & lookup table setup

3. Module 3 – Summary Table Creation

- Business transformations & aggregations

Supporting Components

- Unity Catalog (catalog + schema)
- Databricks Volume (file management)
- ETL execution logs
- Archival handling

1.7 Governance & Data Management Philosophy

This project enforces **enterprise-grade data governance concepts**, including:

- **Centralized metadata management** using Unity Catalog
- **Partitioned Delta tables** for performance
- **Audit columns** for compliance
- **Execution logging** for observability
- **Controlled file movement** via archival rules

These practices reflect real-world data platform standards used in production environments.

1.8 Business Relevance

The final zomato_summary table supports:

- Country-wise restaurant analytics
- Cuisine classification
- Rating band analysis
- Price range insights
- Data quality-filtered reporting

This mirrors how companies operationalize third-party datasets for **business intelligence and decision-making**.

2. Environment Setup

This section describes the complete **Databricks environment initialization** required for the **Zomato ETL Implementation**.

All objects are created using **Unity Catalog**, ensuring governance, consistency, and production-grade table management.

The environment setup is implemented as a **dedicated Databricks notebook** titled:

ZOMATO ENVIRONMENT SETUP NOTEBOOK

This notebook is designed to be executed **once** before running the ETL pipeline.

2.1 Unity Catalog Configuration

Unity Catalog is used as the centralized governance layer for managing:

- Catalogs
- Schemas
- Delta tables
- Volumes (file storage)

All project assets are organized under a **single catalog and schema** to maintain logical isolation.

Catalog Created

```
1 zomato
```

Schema Created

```
1 etl
```

Purpose

- *zomato* catalog groups all Zomato-related objects
- *etl* schema contains ETL tables, logs, and intermediate layers

Implementation Code

```
1 spark.sql("CREATE CATALOG IF NOT EXISTS zomato")
2 spark.sql("CREATE SCHEMA IF NOT EXISTS zomato.etl")
3
4 spark.sql("USE CATALOG zomato")
5 spark.sql("USE SCHEMA etl")
```

✅ Ensures catalog and schema exist before table or volume creation.

2.2 Volume Creation

Volume Name

```
1 zomato_files
```

Purpose of Volume

Databricks Volumes provide:

- Governed storage managed by Unity Catalog

- Secure and consistent access path
- Elimination of direct DBFS dependency

All inbound data, processed files, archives, and logs are stored inside this volume.

Implementation Code

```
1 spark.sql("""
2 CREATE VOLUME IF NOT EXISTS zomato_files
3 """)
```

Resolved Volume Path

```
1 /Volumes/zomato/etl/zomato_files
```

2.3 Volume Folder Structure Setup

To support structured file movement and ETL stages, the following directory layout is created inside the volume.

Folder Structure

```
1 zomato_files/
2 |
3 |— zomato_raw_files/
4 |
5 |— zomato_etl/
6 | |— source/
7 | | |— json/
8 | | |— csv/
9 | | |— archive/
10 | |
11 | |— logs/
```

Purpose of Each Folder

Folder	Description
--------	-------------

zomato_raw_files	Initial landing zone for incoming JSON files
source/json	JSON files selected for ETL processing
source/csv	CSV files generated after JSON conversion
source/archive	Successfully processed JSON files
logs	ETL execution logs (optional file-based logging)

Implementation Code

```

1  base_path = "/Volumes/zomato/etl/zomato_files"
2
3  folders = [
4      f"{base_path}/zomato_raw_files",
5      f"{base_path}/zomato_etl/source/json",
6      f"{base_path}/zomato_etl/source/csv",
7      f"{base_path}/zomato_etl/source/archive",
8      f"{base_path}/zomato_etl/logs"
9  ]
10
11  for folder in folders:
12      dbutils.fs.mkdirs(folder)

```

✅ Ensures folder structure exists before ETL execution.

2.4 Raw Table Creation – *zomato_raw*

Table Name

```
1  zomato.etl.zomato_raw
```

Purpose

This table represents the **Bronze layer** of the medallion architecture.

It stores:

- Structured raw Zomato data

- Minimal transformations
- One-to-one mapping from CSV files

Key Characteristics

- Delta Lake table
- Partitioned by *filedate*
- Supports incremental ingestion
- New partitions created automatically

Table Definition

```
1  spark.sql("""
2  CREATE TABLE IF NOT EXISTS zomato.etl.zomato_raw
3  (
4  restaurant_id STRING,
5  restaurant_name STRING,
6  country_code INT,
7  city STRING,
8  address STRING,
9  locality STRING,
10 locality_verbose STRING,
11 longitude DOUBLE,
12 latitude DOUBLE,
13 cuisines STRING,
14 average_cost_for_two INT,
15 currency STRING,
16 has_table_booking INT,
17 has_online_delivery INT,
18 is_delivering_now INT,
19 switch_to_order_menu INT,
20 price_range INT,
```



```

21  aggregate_rating DOUBLE,
22  rating_text STRING,
23  votes INT,
24  filedate DATE
25  )
26  USING DELTA
27  PARTITIONED BY (filedate)
28  "")

```

2.5 Dimension Table Creation – *dim_country*

Table Name

```
1  zomato.etl.dim_country
```

Purpose

This table stores country reference data and is used for:

- Lookup joins
- Enriching raw Zomato records
- Deriving partition column *p_country_name*

Table Definition

```

1  spark.sql("""
2  CREATE TABLE IF NOT EXISTS dim_country (
3    country_code INT,
4    country_name STRING
5  )
6  USING DELTA
7  """)

```

2.6 Summary Table Creation – *zomato_summary*

Table Name

1 zomato.etl.zomato_summary

Purpose

This table represents the **Gold layer** and contains:

- Cleaned
- Enriched
- Business-ready Zomato data

Partitioning Strategy

- *p_filedate*
- *p_country_name*

This ensures:

- Query performance optimization
- Country-wise and date-wise filtering

Table Definition

```
1 spark.sql("""
2 CREATE TABLE IF NOT EXISTS zomato_summary (
3     restaurant_id STRING,
4     restaurant_name STRING,
5     country_code INT,
6     city STRING,
7     address STRING,
8     locality STRING,
9     locality_verbose STRING,
10    longitude DOUBLE,
11    latitude DOUBLE,
12    cuisines STRING,
13    average_cost_for_two INT,
14    currency STRING,
15    has_table_booking STRING,
```

```

16   has_online_delivery STRING,
17   is_delivering_now STRING,
18   switch_to_order_menu STRING,
19   price_range INT,
20   aggregate_rating DOUBLE,
21   rating_text STRING,
22   votes INT,
23   m_rating_colour STRING,
24   m_cuisines STRING,
25   create_datetime TIMESTAMP,
26   user_id STRING,
27   p_filedate DATE,
28   p_country_name STRING
29 )
30 USING DELTA
31 PARTITIONED BY (p_filedate, p_country_name)
32 """)

```

2.7 Logging Table Creation – *zomato_summary_log*

Purpose

This table captures **ETL execution metadata** for:

- Audit
- Monitoring
- Failure analysis

Table Definition

```

1  spark.sql("""
2  CREATE TABLE IF NOT EXISTS zomato_summary_log
3  (

```

```
4  job_id STRING,  
5  job_step STRING,  
6  spark_submit_command STRING,  
7  job_start_time TIMESTAMP,  
8  job_end_time TIMESTAMP,  
9  job_status STRING  
10 )  
11 USING DELTA  
12 """)
```

2.8 Environment Validation Criteria

The environment setup is considered **successfully completed** when:

- ✓ Catalog *zomato* exists
- ✓ Schema *etl* exists
- ✓ Volume *zomato_files* exists
- ✓ Folder structure is created
- ✓ Delta tables are available
- ✓ No errors during notebook execution

3. Module 1 – JSON to CSV Conversion

3.1 Module Objective

The objective of **Module 1** is to convert incoming **Zomato JSON files** into **structured CSV files** that can be reliably ingested into Delta Lake tables.

This module handles:

- File movement and staging
- JSON parsing and normalization
- Column extraction based on business requirements
- Filedate derivation

- CSV generation with standardized naming
- Archival of processed JSON files

3.2 Source Data Overview

Source Format

- JSON
- Nested structure
- Array of restaurant objects

Landing Location

1 /Volumes/zomato/etl/zomato_files/zomato_raw_files

Raw JSON files are assumed to arrive in this folder and are **not processed directly**. They are first copied to a controlled processing area.

3.3 Folder Preparation & Path Initialization

Before processing begins, all standard paths are initialized to ensure consistency across notebooks.

Paths Used

- 1 raw_path = "/Volumes/zomato/etl/zomato_files/zomato_raw_files"
- 2 json_path = "/Volumes/zomato/etl/zomato_files/zomato_etl/source/json"
- 3 csv_path = "/Volumes/zomato/etl/zomato_files/zomato_etl/source/csv"
- 4 archive_path = "/Volumes/zomato/etl/zomato_files/zomato_etl/source/archive"

These paths correspond to the **controlled ETL folder structure** created in the environment setup.

3.4 Identify Incoming JSON Files

The pipeline dynamically identifies all JSON files present in the raw landing directory.

Implementation Logic

- 1 files = dbutils.fs.ls(raw_path)
- 2 json_files = [file for file in files if file.name.endswith(".json")]

3 json_files

✓ This ensures:

- Only *.json* files are selected
- New files are picked automatically
- No hard-coding of filenames

3.5 Copy JSON Files to Processing Directory

To avoid accidental reprocessing or corruption of raw data, files are **copied (not moved)** into the processing directory.

Source → Destination

1 zomato_raw_files → zomato_etl/source/json

Implementation Code

```
1 for file in json_files:
2     source = file.path
3     destination = f"{json_path}/{file.name}"
4
5     try:
6         dbutils.fs.cp(source, destination)
7
8         files_in_dest = dbutils.fs.ls(json_path)
9         copied_files = [f.name for f in files_in_dest]
10
11         if file.name in copied_files:
12             print(f"SUCCESS: {file.name} copied to source/json")
13         else:
14             print(f"ERROR: {file.name} was NOT copied to source/json")
15
16     except Exception as e:
```

```
17     print(f"FAILED: Could not copy {file.name}")
18     print(f"Error: {e}")
```

✅ Built-in verification confirms successful copy.

3.6 Validate JSON Files Ready for Processing

Before transformation begins, the processing folder is scanned again.

```
1  files = dbutils.fs.ls(json_path)
2  json_files = [file for file in files if file.name.endswith(".json")]
3  json_files
```

This ensures:

- Files exist in the correct folder
- Downstream logic operates only on staged files

3.7 Filedate Derivation Logic

Each file is tagged with a **filedate**, which later:

- Drives table partitioning
- Enables incremental processing
- Supports backfills

Rule

- Extract *YYYY-MM-DD* from filename
- If not present, use current date

Implementation

```
1  import re
2  from datetime import datetime
3
4  match = re.search(r'\d{4}-\d{2}-\d{2}', original_name)
5
6  if match:
```

```
7     filedate = match.group(0)
8     else:
9         filedate = datetime.now().strftime('%Y-%m-%d')
```

3.8 JSON Parsing & Flattening

JSON Structure

- Root object contains *restaurants* array
- Each array element wraps a *restaurant* object

Flattening Steps

1. Read multiline JSON
2. Explode *restaurants* array
3. Extract nested restaurant attributes

Implementation

```
1 df = spark.read.option("multiline", "true").json(file_path)
2
3 df_res = df.select(explode(col("restaurants")).alias("restaurant_wrapper"))
4 df_res = df_res.select(col("restaurant_wrapper.restaurant").alias("restaurant"))
```

3.9 Column Extraction (20 Mandatory Fields)

Only **business-approved fields** are retained, as defined in the assignment.

Extracted Columns

```
1 df_final = df_res.select(
2     col("restaurant.R.res_id").alias("restaurant_id"),
3     col("restaurant.name").alias("restaurant_name"),
4     col("restaurant.location.country_id").alias("country_code"),
5     col("restaurant.location.city").alias("city"),
6     col("restaurant.location.address").alias("address"),
7     col("restaurant.location.locality").alias("locality"),
```



```

8  col("restaurant.location.locality_verbose").alias("locality_verbose"),
9  col("restaurant.location.longitude").alias("longitude"),
10 col("restaurant.location.latitude").alias("latitude"),
11 col("restaurant.cuisines").alias("cuisines"),
12 col("restaurant.average_cost_for_two").alias("average_cost_for_two"),
13 col("restaurant.currency").alias("currency"),
14 col("restaurant.has_table_booking").alias("has_table_booking"),
15 col("restaurant.has_online_delivery").alias("has_online_delivery"),
16 col("restaurant.is_delivering_now").alias("is_delivering_now"),
17 col("restaurant.switch_to_order_menu").alias("switch_to_order_menu"),
18 col("restaurant.price_range").alias("price_range"),
19 col("restaurant.user_rating.aggregate_rating").alias("aggregate_rating"),
20 col("restaurant.user_rating.rating_text").alias("rating_text"),
21 col("restaurant.user_rating.votes").alias("votes")
22 )

```

3.10 Add Filedate Column

The derived filedate is added as a **date column**.

```

1  df_final = df_final.withColumn(
2    "filedate",
3    to_date(lit(filedate))
4  )

```

3.11 CSV File Generation

CSV Rules

- Single CSV per filedate
- Header enabled
- Naming convention enforced

Naming Standard

```
1 zomato_<filedate>.csv
```

Implementation

```
1 temp_path = f"{csv_path}/temp_{filedate}"
2
3 df_final.coalesce(1) \
4     .write \
5     .mode("overwrite") \
6     .option("header", True) \
7     .csv(temp_path)
```

3.12 Rename & Finalize CSV Output

Spark generates system-named CSV files, so renaming is required.

```
1 files = dbutils.fs.ls(temp_path)
2
3 for f in files:
4     if f.name.endswith(".csv"):
5         final_csv = f"{csv_path}/zomato_{filedate}.csv"
6         dbutils.fs.mv(f.path, final_csv)
```

Temporary folders are then removed:

```
1 dbutils.fs.rm(temp_path, True)
```

✅ Final CSV is now ready for loading into Delta tables.

3.13 Archive Processed JSON Files

After successful CSV creation, the source JSON files are **archived**.

Archival Rule

```
1 source/json → source/archive
```

Implementation

```
1 for file in json_files:
2     dbutils.fs.mv(
3         file.path,
4         f"{archive_path}/{file.name}"
5     )
```

✓ This prevents duplicate processing and maintains clear lineage.

3.14 Module 1 Completion Criteria

Module 1 is considered successful when:

- ✓ JSON files are copied from raw to source
- ✓ CSV files created with correct naming
- ✓ Mandatory 20 fields extracted
- ✓ Filedate correctly derived
- ✓ JSON files archived
- ✓ No unprocessed files remain in source/json

4. Module 2 – Raw Table & Dimension Table Creation and Loading

4.1 Module Objective

The objective of **Module 2** is to load structured CSV data (generated in Module 1) into **Unity Catalog–managed Delta Lake tables**.

This module is responsible for:

- Creating raw and dimension tables in Unity Catalog
- Loading CSV data into the raw Delta table
- Implementing **incremental load logic**
- Handling **partition-level overwrites**
- Loading and maintaining the country reference dimension

This module represents the **Bronze Layer ingestion completion** and prepares data for downstream transformations.

4.2 Input Data Sources

CSV Data (From Module 1)

```
1 /Volumes/zomato/etl/zomato_files/zomato_etl/source/csv
```

File pattern:

```
1 zomato_<filedate>.csv
```

Country Dimension Reference

```
1 /Volumes/zomato/etl/zomato_files/zomato_raw_files/country_code.csv
```

4.3 Unity Catalog Context Initialization

All operations are executed within the governed Unity Catalog namespace.

Notebook Initialization

```
1 spark.sql("USE CATALOG zomato")
```

```
2 spark.sql("USE SCHEMA etl")
```

```
3
```

```
4 print("Using schema zomato.etl")
```

✅ Ensures all tables are created and accessed under the correct catalog and schema.

4.4 Raw Table Creation – *zomato_raw*

Table Purpose

The *zomato_raw* table stores **structured raw Zomato data** with:

- Minimal transformation
- One-to-one mapping from CSV
- Filedate-based partitioning

This table serves as the **Bronze layer**.

Table Properties

- Format: Delta Lake
- Managed by Unity Catalog

- Partitioned by *filedate*

Table Creation SQL

```
1 CREATE TABLE IF NOT EXISTS zomato.etl.zomato_raw
2 (
3     restaurant_id STRING,
4     restaurant_name STRING,
5     country_code INT,
6     city STRING,
7     address STRING,
8     locality STRING,
9     locality_verbose STRING,
10    longitude DOUBLE,
11    latitude DOUBLE,
12    cuisines STRING,
13    average_cost_for_two INT,
14    currency STRING,
15    has_table_booking INT,
16    has_online_delivery INT,
17    is_delivering_now INT,
18    switch_to_order_menu INT,
19    price_range INT,
20    aggregate_rating DOUBLE,
21    rating_text STRING,
22    votes INT,
23    filedate DATE
24 )
25 USING DELTA
26 PARTITIONED BY (filedate)
```

4.5 CSV Data Reading with Schema Enforcement

To avoid schema drift and ensure consistency, CSV files are read using the **existing table schema**.

Implementation

```
1 df_csv = (  
2     spark.read  
3     .schema(spark.table("zomato_raw").schema)  
4     .option("header", True)  
5     .csv(f"{csv_path}/zomato_*.csv")  
6 )
```

✅ This ensures:

- Column type consistency
- Safer ingestion
- Alignment with Delta table definition

4.6 Filedate Validation and Default Handling

To guarantee valid partitioning, the *filedate* column is always populated.

Logic

- Use filedate from CSV if present
- Otherwise default to current system date

Implementation

```
1 from pyspark.sql.functions import col, current_date, coalesce  
2  
3 df_csv = df_csv.withColumn(  
4     "filedate",  
5     coalesce(col("filedate"), current_date())  
6 )
```

4.7 Incremental Load Strategy (Partition-Level Refresh)

Instead of truncating the entire table, the module performs **targeted partition overwrites**.

Process

1. Identify distinct *filedate* values in incoming data
2. Delete existing records for those partitions
3. Append fresh data

Implementation

Extract Filedates

```
1 filedates = [row.filedate for row in df_csv.select("filedate").distinct().collect()]
```

Delete Existing Partitions

```
1 for fd in filedates:
2     spark.sql(f"""
3         DELETE FROM zomato.etl.zomato_raw
4         WHERE filedate = DATE('{fd}')
5     """)
```

Append New Data

```
1 df_csv.write \
2     .format("delta") \
3     .mode("append") \
4     .saveAsTable("zomato.etl.zomato_raw")
```

✅ This approach ensures:

- Idempotent processing
- Safe daily reruns
- No duplicate data
- Efficient incremental loads

4.8 Raw Table Validation

After loading, the raw table is validated visually.

```
1 spark.table("zomato_raw").display()
```

4.9 Dimension Table Creation – *dim_country*

Purpose

The *dim_country* table provides reference data used to:

- Enrich Zomato records
- Derive country names
- Support joins in Module 3

Table Creation

```
1 CREATE TABLE IF NOT EXISTS dim_country (  
2   country_code INT,  
3   country_name STRING  
4 )  
5 USING DELTA
```

4.10 Load Country Dimension Data

Read Country CSV

```
1 df_country = (  
2   spark.read  
3   .option("header", True)  
4   .csv(country_path)  
5 )
```

Column Standardization

The raw CSV column names are standardized to align with warehouse schema.

```
1 df_country_clean = df_country.select(
```



```
2    col("Country Code").cast("int").alias("country_code"),
3    col("Country").alias("country_name")
4 )
```

Overwrite Dimension Table

The country dimension is fully refreshed on each run.

```
1 df_country_clean.write \
2   .mode("overwrite") \
3   .format("delta") \
4   .saveAsTable("dim_country")
```

✅ Overwrite mode is acceptable as:

- Dimension size is small
- Reference data changes infrequently

4.11 Dimension Validation

```
1 spark.table("dim_country").show()
```

4.12 Metadata Validation

To verify all tables created successfully:

```
1 spark.sql("SHOW TABLES IN zomato.etl").show()
```

4.13 Module 2 Completion Criteria

Module 2 is considered successful when:

- ✅ *zomato_raw* table exists and is populated
- ✅ Data is partitioned by *filedate*
- ✅ Incremental overwrite logic works correctly
- ✅ *dim_country* table is loaded
- ✅ Schema consistency is maintained
- ✅ No duplicate raw records exist

5. Module 3 – Summary Table Creation & Transformations

5.1 Module Objective

The objective of **Module 3** is to transform raw Zomato data into an **analytics-ready summary table** that applies all required **business rules, data quality checks, enrichments, and audit columns**.

This module represents the **Gold Layer** of the Medallion Architecture and produces the final dataset for reporting and analysis.

5.2 Input Tables

Source Tables

Table	Purpose
<i>zomato_raw</i>	Structured raw restaurant data
<i>dim_country</i>	Country reference / lookup table

5.3 Load Raw & Dimension Data

The transformation pipeline begins by loading the raw and dimension tables into Spark DataFrames.

```
1 df_raw = spark.table("zomato_raw")
2 df_country = spark.table("dim_country")
```

These DataFrames serve as the base for all downstream transformations.

5.4 Join with Country Dimension (Mandatory Rule)

Business Requirement

- Join Zomato data with country reference data
- Join condition: *country_code*
- Derive *country_name*

Implementation

```
1 from pyspark.sql.functions import col
2
3 df_joined = df_raw.join(
4     df_country,
5     df_raw.country_code == df_country.country_code,
6     "left"
7 ).select(
8     df_raw["*"],
9     df_country["country_name"]
10 )
```

✅ A **LEFT JOIN** is used to prevent accidental data loss if a country code is missing in the dimension.

5.5 Filter Rule – Remove Invalid Cuisines (Mandatory Rule)

Business Requirement

Remove records where:

- *cuisines* is NULL
- *cuisines* is blank

Implementation

```
1 df_filtered = df_joined.filter(
2     (col("cuisines").isNotNull()) &
3     (col("cuisines") != "")
4 )
```

✅ Ensures only analytically meaningful restaurant records are retained.

5.6 Null Handling for String Columns (Mandatory Rule)

Business Requirement

For **all string columns**:

- Replace NULL values with "NA"

Implementation

```
1 string_cols = [  
2     f.name for f in df_filtered.schema.fields  
3     if f.dataType.simpleString() == 'string'  
4 ]  
5  
6 df_clean = df_filtered.fillna("NA", subset=string_cols)
```

- ✓ Enforces consistent string representation across the dataset.

5.7 Deduplication Logic (Mandatory Rule)

Business Key Definition

A Zomato restaurant record is uniquely identified by:

- *restaurant_id*
- *filedate*

Implementation

```
1 df_dedup = df_clean.dropDuplicates(["restaurant_id", "filedate"])
```

- ✓ Prevents duplicate metrics across repeated loads or reprocessing runs.

5.8 Derive *m_rating_colour* (Mandatory Rule)

Business Logic

Aggregate Rating Range	Colour
1.9 – 2.4	Red
2.5 – 3.4	Amber
3.5 – 3.9	Light Green
4.0 – 4.4	Green

4.5 – 5.0	Gold
Else	NA

Implementation

```

1  from pyspark.sql.functions import when
2
3  df_rating = df_dedup.withColumn(
4      "m_rating_colour",
5      when(col("aggregate_rating").between(1.9,2.4), "Red")
6      .when(col("aggregate_rating").between(2.5,3.4), "Amber")
7      .when(col("aggregate_rating").between(3.5,3.9), "Light Green")
8      .when(col("aggregate_rating").between(4.0,4.4), "Green")
9      .when(col("aggregate_rating").between(4.5,5.0), "Gold")
10     .otherwise("NA")
11 )

```

5.9 Derive *m_cuisines* (Mandatory Rule)

Business Logic

- If cuisines contain Indian keywords → *"Indian"*
- Otherwise → *"World Cuisines"*

Implementation

```

1  from pyspark.sql.functions import lower
2
3  df_cuisine = df_rating.withColumn(
4      "m_cuisines",
5      when(lower(col("cuisines")).contains("indian"), "Indian")
6      .otherwise("World Cuisines")
7  )

```

5.10 Add Audit Columns (Mandatory Rule)

Audit Fields

- *create_datetime* → record creation timestamp
- *user_id* → ETL execution user

Implementation

```
1 from pyspark.sql.functions import current_timestamp, lit
2
3 df_audit = df_cuisine.withColumn(
4     "create_datetime", current_timestamp()
5 ).withColumn(
6     "user_id", lit("etl_user")
7 )
```

✅ Supports lineage, compliance, and audit use cases.

5.11 Derive Partition Columns

Required Partition Columns

- *p_filedate*
- *p_country_name*

Implementation

```
1 df_summary = df_audit.withColumnRenamed(
2     "filedate", "p_filedate"
3 ).withColumnRenamed(
4     "country_name", "p_country_name"
5 )
```

5.12 Summary Table Definition

Target Table

```
1 zomato.etl.zomato_summary
```

Partition Strategy

- Partitioned by *p_filedate*
- Partitioned by *p_country_name*

Table Creation SQL

```
1 CREATE TABLE IF NOT EXISTS zomato_summary (  
2     restaurant_id STRING,  
3     restaurant_name STRING,  
4     country_code INT,  
5     city STRING,  
6     address STRING,  
7     locality STRING,  
8     locality_verbose STRING,  
9     longitude DOUBLE,  
10    latitude DOUBLE,  
11    cuisines STRING,  
12    average_cost_for_two INT,  
13    currency STRING,  
14    has_table_booking STRING,  
15    has_online_delivery STRING,  
16    is_delivering_now STRING,  
17    switch_to_order_menu STRING,  
18    price_range INT,  
19    aggregate_rating DOUBLE,  
20    rating_text STRING,  
21    votes INT,  
22    m_rating_colour STRING,  
23    m_cuisines STRING,  
24    create_datetime TIMESTAMP,
```

```

25     user_id STRING,
26     p_filedate DATE,
27     p_country_name STRING
28 )
29 USING DELTA
30 PARTITIONED BY (p_filedate, p_country_name)

```

5.13 Data Type Alignment Before Load

To match the target schema, integer flag columns are cast to strings.

```

1 df_summary = df_summary.withColumn(
2     "has_table_booking", col("has_table_booking").cast("string")
3 ).withColumn(
4     "has_online_delivery", col("has_online_delivery").cast("string")
5 ).withColumn(
6     "is_delivering_now", col("is_delivering_now").cast("string")
7 ).withColumn(
8     "switch_to_order_menu", col("switch_to_order_menu").cast("string")
9 )

```

5.14 Load Summary Table

Load Strategy

- Append mode
- Partition-based storage
- Suitable for both historical and incremental loads

Implementation

```

1 df_summary.write \
2     .mode("append") \
3     .saveAsTable("zomato_summary")

```


5.15 Validation & Sample Queries

```
1 spark.table("zomato_summary").show(20)
1 SELECT * FROM zomato_summary LIMIT 20;
```

✔ Confirms data availability and correctness.

5.16 Module 3 Completion Criteria

Module 3 is considered successful when:

- ✔ All mandatory transformations are applied
 - ✔ Dimension enrichment is correct
 - ✔ Data quality rules enforced
 - ✔ Deduplication applied
 - ✔ Partition columns populated
 - ✔ Audit columns present
 - ✔ Summary table populated
-

6. Module 5 – Load Strategies

This section explains the supported **data loading strategies** implemented in the Zomato ETL pipeline.

The system supports both **manual selective loading** and **full historical loading**, enabling operational flexibility.

6.1 Manual Load Strategy

6.1.1 Objective

The **Manual Load strategy** allows users to selectively load data into the *zomato_summary* table based on:

- A specific *filedate*
- A specific *country_code*
- Or a combination of both

This mode is particularly useful for:

- Backfills
- Reprocessing corrected data
- Country-specific or date-specific reloads
- Debugging and validation

6.1.2 Notebook Description

This logic is implemented in a dedicated notebook titled:

ZOMATO MANUAL LOAD NOTEBOOK

The notebook is **parameter-driven** and reuses all transformation rules defined in **Module 3**, ensuring consistency across load modes.

6.1.3 Environment Initialization

Before executing transformations, the notebook explicitly sets the Unity Catalog context.

```
1 catalog = "zomato"
2 schema = "etl"
3
4 spark.sql(f"USE CATALOG {catalog}")
5 spark.sql(f"USE SCHEMA {schema}")
```

✅ Ensures all tables are accessed under the governed namespace.

6.1.4 Runtime Parameters (Widgets)

Databricks widgets are used to accept user input during execution.

Input Parameters

Parameter	Description
<i>filedate</i>	Optional date filter (YYYY-MM-DD)
<i>country_code</i>	Optional country filter

Widget Definition

```
1 dbutils.widgets.text("filedate","")
2 dbutils.widgets.text("country_code","")
3
4 filedate = dbutils.widgets.get("filedate")
5 country_code = dbutils.widgets.get("country_code")
```

✓ Parameters are optional — if left empty, no filter is applied.

6.1.5 Read Raw Data

The raw Delta table is read as the base dataset.

```
1 df = spark.table("zomato_raw")
```

6.1.6 Apply Manual Filters

Filters are applied **conditionally**, only when parameter values are provided.

```
1 if filedate:
2     df = df.filter(col("filedate") == filedate)
3
4 if country_code:
5     df = df.filter(col("country_code") == country_code)
```

✓ This allows flexible execution modes:

- Date-only load
- Country-only load
- Date + country load

6.1.7 Join with Country Dimension

The filtered raw data is enriched with country names using the dimension table.

```
1 df = df.join(
2     spark.table("dim_country"),
3     "country_code",
```

```
4     "left"
```

```
5 )
```

✅ LEFT join ensures no raw data loss.

6.1.8 Apply Data Quality Rules

Filter Invalid Cuisines

```
1 df = df.filter(
```

```
2     (col("cuisines").isNotNull()) &
```

```
3     (col("cuisines") != ""))
```

```
4 )
```

Null Handling for String Columns

```
1 string_cols = [
```

```
2     f.name for f in df.schema.fields
```

```
3     if f.dataType.simpleString() == "string"
```

```
4 ]
```

```
5
```

```
6 df = df.fillna("NA", subset=string_cols)
```

6.1.9 Deduplication

Records are deduplicated based on the defined business key.

```
1 df = df.dropDuplicates(["restaurant_id","filedate"])
```

✅ Prevents duplicate summary records during reprocessing.

6.1.10 Derive Business Columns

Rating Colour

```
1 df = df.withColumn(
```

```
2     "m_rating_colour",
```

```

3   when(col("aggregate_rating").between(1.9,2.4),"Red")
4   .when(col("aggregate_rating").between(2.5,3.4),"Amber")
5   .when(col("aggregate_rating").between(3.5,3.9),"Light Green")
6   .when(col("aggregate_rating").between(4.0,4.4),"Green")
7   .when(col("aggregate_rating").between(4.5,5.0),"Gold")
8   .otherwise("NA")
9 )

```

Cuisine Classification

```

1  df = df.withColumn(
2    "m_cuisines",
3    when(lower(col("cuisines")).contains("indian"),"Indian")
4    .otherwise("World Cuisines")
5  )

```

6.1.11 Add Audit Columns

Audit metadata is added to track manual executions.

```

1  df = df.withColumn("create_datetime", current_timestamp()) \
2    .withColumn("user_id", lit("manual_load"))

```

 Distinguishes manual runs from scheduled ETL executions.

6.1.12 Prepare Partition Columns

Columns are renamed to match the summary table partition strategy.

```

1  df = df.withColumnRenamed("filedate","p_filedate") \
2    .withColumnRenamed("country_name","p_country_name")

```

6.1.13 Load into Summary Table

The transformed data is appended into the summary table.

```
1 df.write \
2 .mode("append") \
3 .insertInto("zomato_summary")
```

- ✔ Supports partial, targeted loading without impacting existing data.

6.1.14 Completion Message

```
1 print("Manual Load Completed Successfully")
```

6.1.15 Manual Load Use-Case Examples

Scenario	Parameters
Load a specific day	<i>filedate = 2023-04-01</i>
Load a specific country	<i>country_code = 1</i>
Targeted backfill	Both parameters
Full load using manual notebook	Leave both empty

6.1.16 Manual Load Completion Criteria

Manual Load is considered successful when:

- ✔ Parameters applied correctly
- ✔ Filters behave dynamically
- ✔ All transformation rules are applied
- ✔ Data appended to *zomato_summary*
- ✔ Audit columns reflect *manual_load*

6.2 Historical Load Strategy

6.2.1 Objective

The **Historical Load strategy** enables a **full end-to-end reload** of the Zomato summary dataset using **all available raw data** across:

- All filedates

- All countries

This mode is designed for:

- Initial data population
- Full reprocessing after logic changes
- Recovery scenarios
- Data correctness revalidation

Unlike the manual load, the historical load **ignores runtime filters** and recreates the summary table from scratch.

6.2.2 Notebook Description

This strategy is implemented in a dedicated notebook titled:

ZOMATO HISTORICAL LOAD NOTEBOOK

The notebook reuses the **same transformation rules** as Module 3 and Manual Load, ensuring **logical consistency** across all execution modes.

6.2.3 Environment Initialization

Before execution, the Unity Catalog context is explicitly set.

```
1 catalog = "zomato"
2 schema = "etl"
3
4 spark.sql(f"USE CATALOG {catalog}")
5 spark.sql(f"USE SCHEMA {schema}")
```

✓ Ensures all reads and writes occur within the governed *zomato.etl* namespace.

6.2.4 Read Complete Raw Dataset

The historical load begins by reading the entire raw Delta table.

```
1 df = spark.table("zomato_raw")
```

✓ No filters are applied at this stage, allowing full dataset processing.

6.2.5 Enrich with Country Dimension

To derive country names and enable partitioning, the raw data is enriched using the country dimension.

```
1 df = df.join(  
2     spark.table("dim_country"),  
3     "country_code",  
4     "left"  
5 )
```

✓ LEFT join ensures no loss of raw data due to missing dimension records.

6.2.6 Apply Data Quality Filter (Mandatory Rule)

Business Rule

Remove records where:

- *cuisines* is NULL
- *cuisines* is blank

```
1 df = df.filter(  
2     (col("cuisines").isNotNull()) &  
3     (col("cuisines") != "")  
4 )
```

✓ Ensures only analytically valid restaurant records are retained.

6.2.7 Null Handling for String Columns

Business Rule

- Replace NULL values with "NA" for all string columns

```
1 string_cols = [  
2     f.name for f in df.schema.fields  
3     if f.dataType.simpleString() == "string"
```



```
4 ]
```

```
5
```

```
6 df = df.fillna("NA", subset=string_cols)
```

✅ Enforces consistent data representation across the dataset.

6.2.8 Deduplication Logic

Business Key

- *restaurant_id*
- *filedate*

```
1 df = df.dropDuplicates(["restaurant_id","filedate"])
```

✅ Prevents duplicate records caused by reprocessing or repeated ingestions.

6.2.9 Derive *m_rating_colour*

Rating Band Logic

Rating Range	Colour
1.9 – 2.4	Red
2.5 – 3.4	Amber
3.5 – 3.9	Light Green
4.0 – 4.4	Green
4.5 – 5.0	Gold
Else	NA

```
1 df = df.withColumn(
```

```
2     "m_rating_colour",
```

```
3     when(col("aggregate_rating").between(1.9,2.4),"Red")
```

```
4     .when(col("aggregate_rating").between(2.5,3.4),"Amber")
```

```
5     .when(col("aggregate_rating").between(3.5,3.9),"Light Green")
```

```
6     .when(col("aggregate_rating").between(4.0,4.4),"Green")
```

```

7     .when(col("aggregate_rating").between(4.5,5.0),"Gold")
8     .otherwise("NA")
9 )

```

6.2.10 Derive *m_cuisines*

Cuisine Classification Rule

- Contains “indian” → *Indian*
- Otherwise → *World Cuisines*

```

1 df = df.withColumn(
2     "m_cuisines",
3     when(lower(col("cuisines")).contains("indian"),"Indian")
4     .otherwise("World Cuisines")
5 )

```

6.2.11 Add Audit Columns

Audit Fields

Column	Purpose
<i>create_datetime</i>	Record creation timestamp
<i>user_id</i>	Execution mode identifier

```

1 df = df.withColumn("create_datetime", current_timestamp()) \
2     .withColumn("user_id", lit("historical_load"))

```

- ✓ Distinguishes historical runs from manual and daily executions.

6.2.12 Prepare Partition Columns

To align with the summary table’s partitioning strategy:

```

1 df = df.withColumnRenamed("filedate","p_filedate") \
2     .withColumnRenamed("country_name","p_country_name")

```

6.2.13 Historical Load Write Strategy

Load Behavior

- Fully rebuilds the summary table
- Overwrites all existing data
- Recreates partitions
- Updates schema if required

```
1 df.write \  
2 .mode("overwrite") \  
3 .option("overwriteSchema", "true") \  
4 .saveAsTable("zomato_summary")
```

✅ Suitable for full reloads and first-time deployments.

6.2.14 Completion Message

```
1 print("Historical Load Completed Successfully")
```

6.2.15 Comparison: Manual vs Historical Load

Aspect	Manual Load	Historical Load
Scope	Partial	Full
Filters	✅ Supported	❌ Not Supported
Mode	Append	Overwrite
Use Case	Backfills	Initial/full reload
user_id	<i>manual_load</i>	<i>historical_load</i>

6.2.16 Historical Load Completion Criteria

Historical Load is successful when:

- ✅ All raw data is processed
- ✅ All transformations are applied
- ✅ Summary table rebuilt fully

- ✓ Partition columns populated
 - ✓ Audit columns reflect historical load
 - ✓ No duplicate records exist
-

7. Orchestration – Driver Notebook

7.1 Objective

The objective of the **Driver Notebook** is to orchestrate the **entire Zomato ETL workflow** in a controlled, auditable, and fault-tolerant manner.

Instead of shell scripts or external schedulers, this project uses a **Databricks Notebook–based orchestration**, which:

- Controls execution flow across modules
- Logs execution status for each step
- Ensures only one ETL run occurs at a time
- Supports multiple load strategies
- Performs archival and log cleanup automatically
- Stops execution immediately on failure

7.2 Driver Notebook Overview

The orchestration logic is implemented in a notebook titled:

ZOMATO DRIVER NOTEBOOK

This notebook acts as a **single entry point** for the entire ETL pipeline.

7.3 Environment Initialization

The Driver Notebook explicitly sets the **Unity Catalog context** to avoid accidental writes outside the governed namespace.

```
1 catalog = "zomato"
2 schema = "etl"
3
4 spark.sql(f"USE CATALOG {catalog}")
```

```
5 spark.sql(f"USE SCHEMA {schema}")
```

✓ Ensures all tables, logs, and files are created under *zomato.etl*.

7.4 Run Environment Setup First

To guarantee idempotency and avoid missing objects, the environment setup notebook is executed at the start of every run.

```
1 dbutils.notebook.run("environment_setup", 0)
```

✓ Ensures:

- Catalog exists
- Schema exists
- Volume exists
- Folder structure exists
- Tables are present

7.5 Runtime Parameters (Widgets)

The Driver Notebook supports **dynamic execution modes** via widgets.

Widgets Defined

```
1 dbutils.widgets.dropdown("load_type","historical",["historical","manual"])
2 dbutils.widgets.text("filedate","")
3 dbutils.widgets.text("country_code","")
```

Parameters

Widget	Purpose
<i>load_type</i>	Select historical or manual load
<i>filedate</i>	Filter for manual load
<i>country_code</i>	Filter for manual load

These parameters are passed downstream to child notebooks.

7.6 Path Configuration

Key ETL paths are initialized centrally to avoid duplication and inconsistencies.

```
1 json_path = "/Volumes/zomato/etl/zomato_files/zomato_etl/source/json"
2 archive_path = "/Volumes/zomato/etl/zomato_files/zomato_etl/source/archive"
3 lock_file = "/Volumes/zomato/etl/zomato_files/zomato_etl/etl.lock"
```

7.7 Job ID Generation

Each pipeline execution is uniquely identified using a UUID.

```
1 import uuid
2 job_id = str(uuid.uuid4())
```

✔ Enables precise execution tracking in logs.

7.8 Centralized Logging Mechanism

Purpose

Execution status for every ETL step is recorded in:

```
1 zomato.etl.zomato_summary_log
```

Logging Function

```
1 def log_step(step, status):
2     spark.sql(f"""
3         INSERT INTO zomato.etl.zomato_summary_log
4         VALUES(
5             '{job_id}',
6             '{step}',
7             '{step} execution',
8             current_timestamp(),
9             current_timestamp(),
10            '{status}'
11        )
12    """)
```

✔ Enables:

- Auditability
- Debugging
- SLA monitoring
- Failure analysis

7.9 Lock Mechanism (Already-Running Protection)

To prevent concurrent runs, a **file-based locking mechanism** is implemented.

Logic

- Check if lock file exists
- If yes → abort execution
- If no → create lock

```
1 try:
2     dbutils.fs.ls(lock_file)
3     raise Exception("ETL is already running")
4 except:
5     dbutils.fs.put(lock_file,"running",True)
```

✔ Ensures **only one ETL instance runs at a time.**

7.10 ETL Execution Flow

All ETL modules are executed inside a **try-except-finally block** to guarantee cleanup.

7.10.1 Job Start Logging

```
1 log_step("JOB_START","STARTED")
```

7.10.2 Module 1 – JSON to CSV

```
1 dbutils.notebook.run("module1_json_to_csv",0)
2 log_step("MODULE1_JSON_TO_CSV","SUCCESS")
```

7.10.3 Module 2 – Raw Table Load

```
1 dbutils.notebook.run("module2_raw_load",0)
2 log_step("MODULE2_RAW_LOAD","SUCCESS")
```

7.10.4 Load Strategy Execution

Manual Load

```
1 dbutils.notebook.run(
2     "manual_load",
3     0,
4     {
5         "filedate":filedate,
6         "country_code":country_code
7     }
8 )
9 log_step("MANUAL_LOAD","SUCCESS")
```

Historical Load

```
1 dbutils.notebook.run("historical_load",0)
2 log_step("HISTORICAL_LOAD","SUCCESS")
```

 Load behavior is driven entirely by runtime widgets.

7.11 Archive Processed JSON Files

After successful processing, all JSON files are archived.

```
1 files = dbutils.fs.ls(json_path)
2
3 for file in files:
4     dbutils.fs.mv(file.path, archive_path + "/" + file.name)
```


- ✓ Prevents reprocessing
- ✓ Maintains data lineage

7.12 Log Retention Policy

Execution logs older than **7 days** are cleaned automatically.

```
1 DELETE FROM zomato.etl.zomato_summary_log
2 WHERE job_start_time < current_timestamp() - INTERVAL 7 DAYS
```

- ✓ Prevents table bloat
- ✓ Meets operational log-retention requirements

7.13 Job Completion Logging

```
1 log_step("JOB_END","SUCCESS")
```

A success message is printed on completion.

7.14 Failure Handling

If any step fails:

```
1 log_step("JOB_FAILED","FAILED")
2 raise e
```

- ✓ Pipeline halts immediately
- ✓ Failure is captured in logs
- ✓ No partial processing

7.15 Lock Release (Finally Block)

Regardless of success or failure, the lock is always released.

```
1 dbutils.fs.rm(lock_file)
```

- ✓ Prevents deadlocks
- ✓ Ensures future runs can proceed

7.16 Orchestration Features Summary

Feature	Implemented
Central control notebook	✓
Step-wise execution	✓
Logging	✓
Lock protection	✓
Dynamic load types	✓
Failure handling	✓
Archival	✓
Log retention	✓

7.17 Orchestration Completion Criteria

The orchestration is considered successful when:

- ✓ All modules execute in correct order
- ✓ Logs are written for each step
- ✓ Load strategy executes based on input
- ✓ JSON files are archived
- ✓ Logs older than 7 days are removed
- ✓ Lock file is cleaned up